

Архитектура процессора, память и NUMA

Вводный блок перед CAS, cache coherence, MESI и sync.Mutex

Go code → ISA → CPU → cache hierarchy → DRAM → NUMA

Зачем backend-разработчику лезть ниже Go

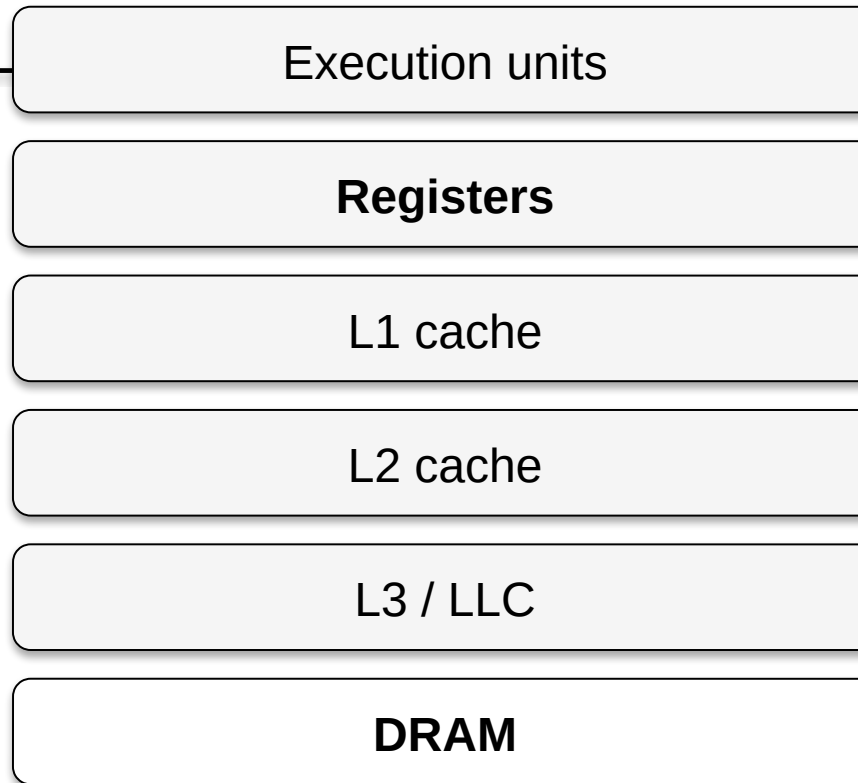
На уровне API `atomic` и `Mutex` выглядят как маленькие операции.

Под нагрузкой стоимость определяется уже не строками Go-кода, а памятью, кешами и взаимодействием ядер.

Чтобы понять contention, false sharing и стоимость CAS, сначала нужна аппаратная модель.

`atomic / Mutex` → `cache line` → `coherence` → `interconnect` → реальная `latency`

Иерархия хранения данных



Чем ближе уровень к execution units — тем ниже latency.

Чем ближе уровень — тем меньше доступный объем.

SSD/HDD находятся уже за пределами непосредственной CPU memory hierarchy.

Регистры: данные уже внутри ядра

```
mov rax, 10  
mov rbx, 20  
add rax, rbx
```

RAX = 30

Регистры — очень маленькое и очень быстрое хранилище непосредственно внутри CPU core.

Execution units работают прежде всего со значениями в регистрах.

Архитектурные регистры видны ISA; физических регистров внутри OoO CPU обычно больше.

архитектурный RAX → physical register 41
следующая версия RAX → physical register 73

Stack / heap и cache — разные уровни модели

**Логическая организация
программы**

stack / heap / globals

Аппаратная иерархия

registers / L1 / L2 / L3 / DRAM

объект из heap может сейчас быть в L1
локальная переменная может остаться только в регистре

CPU cache: зачем он существует

CPU работает в масштабе долей наносекунды; DRAM — десятков наносекунд.
Если каждое чтение ходит в DRAM, ядро тратит сотни тактов на ожидание.
Cache хранит локальные копии недавно и часто используемых участков памяти.

```
DRAM: 0x1000 → 42
      ↓
L1:    0x1000 → 42
```

L1, L2, L3 / LLC

L1

**десятки KiB
несколько cycles**

L2

**сотни KiB – MiB
десятки cycles**

L3 / LLC

**MiB – десятки MiB
десятки–сотни cycles**

L1 обычно разделен на L1I и L1D.

L2 обычно ближе к конкретному core.

L3 часто разделяется между несколькими cores, но реальная topology может быть сложнее.

Cache line — единица перемещения данных



CPU обычно кеширует не отдельный `int64`, а целую cache line.

Типичный размер line на современных x86-64 и многих ARM — 64 bytes.

Эта деталь позже приведет нас к false sharing.

Locality: почему cache вообще работает

Temporal locality

**недавно использованные данные
часто нужны снова**

Spatial locality

**после адреса X часто нужны данные
рядом с X**

```
for i := range values {  
    sum += values[i]  
}
```

Cache hit, miss и write-back

```
L1 hit
  ↓
если miss → L2
  ↓
если miss → L3 / LLC
  ↓
если miss → DRAM
```

Одна и та же load может стоять несколько cycles или несколько сотен cycles.

При write-back CPU может изменить line в cache, а DRAM обновить позже.

Поэтому временно cache может содержать новое значение, а DRAM — старое.

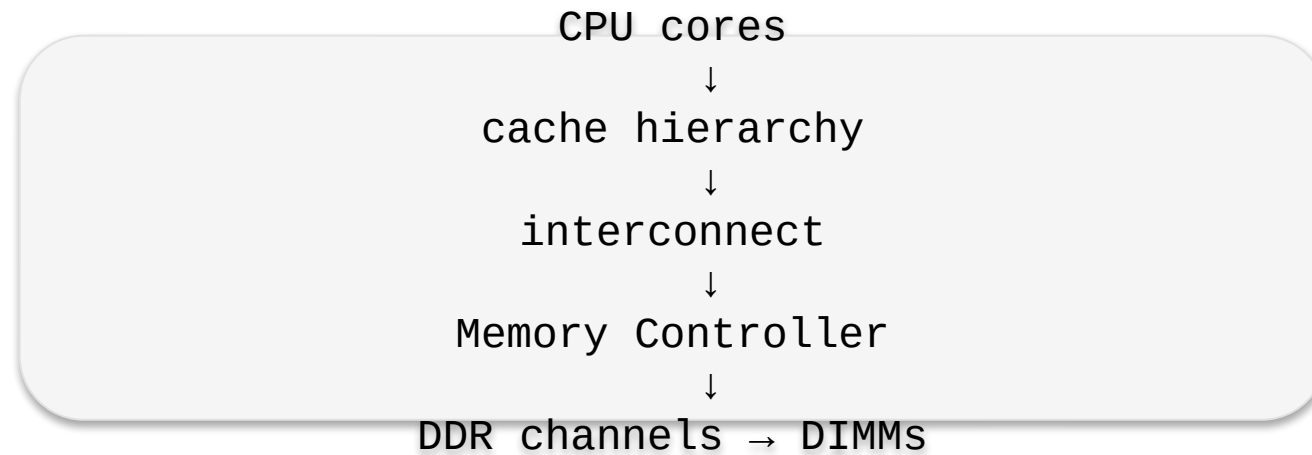
```
cache = новое значение
DRAM  = старое значение
```

DRAM: большая рабочая память

DRAM — Dynamic Random Access Memory: основная оперативная память процессов.

Она плотнее и дешевле SRAM, но заметно медленнее CPU cache.

Ячейки требуют периодического refresh, отсюда Dynamic.



DRAM внутри тоже имеет topology

channel



DIMM



rank



bank



row



column

Для программы RAM выглядит как address → value.

Физически доступ идет через memory controller и структуру DRAM.

Row hit обычно дешевле, чем необходимость переключить row.

Для этой лекции достаточно понимать: DRAM сама не является плоской и одинаково быстрой.

SSD и HDD: это уже storage

NVMe SSD
NAND flash + controller
десятки–сотни μ s

SATA SSD
NAND flash + SATA
сотни μ s

HDD
магнитные пластины +
механика
миллисекунды

SSD / HDD $\rightarrow$ I/O $\rightarrow$ kernel $\rightarrow$ DRAM / page cache $\rightarrow$ CPU caches $\rightarrow$ registers

SSD и HDD — persistent storage, а не еще один уровень CPU cache hierarchy.

Сеть: следующий временной масштаб

process → kernel → socket → NIC → network → remote host → application

Сетевой запрос включает не только передачу байтов, но и работу нескольких программных и аппаратных слоев.

RTT может измеряться от десятков микросекунд в локальной инфраструктуре до десятков и сотен миллисекунд через Internet.

Это уже совершенно другой масштаб по сравнению с CPU cache.

Latency и bandwidth — не одно и то же

Latency

Сколько ждать одну операцию?

Bandwidth

Сколько данных передается за единицу времени?

NVMe может давать несколько GB/s и все равно иметь десятки μ s latency на random read.
100 Gbit/s сеть не отменяет RTT.

Порядок задержек

Операция	Примерная latency	При CPU ~4 GHz
Регистры / простая операция	~0.25–1 ns	~1–4 cycles
L1 hit	~1 ns	~4 cycles
L2 hit	~3–5 ns	~12–20 cycles
L3 / LLC	~10–30 ns	~40–120 cycles
DRAM	~60–120 ns	~240–480 cycles
Remote NUMA DRAM	~100–300+ ns	сотни–1000+ cycles
NVMe random read	~50–200 μ s	сотни тысяч cycles
HDD random access	~5–15 ms	десятки миллионов cycles
Network RTT	~0.1–200+ ms	миллионы–сотни миллионов cycles

Если L1 = 1 секунда

L1	1 секунда
L2	несколько секунд
L3	десятки секунд
DRAM	1–2 минуты
NVMe	около суток
HDD	несколько месяцев
Network	от месяцев до лет

Это не измерительная таблица, а масштабная модель разницы порядков latency.

ISA и microarchitecture

ISA

**что CPU обещает software
инструкции, регистры, addressing,
ordering**

Microarchitecture

**как конкретный CPU это реализует
pipeline, caches, OoO, predictors**

Intel Core и AMD Zen → одна ISA x86-64, но разная microarchitecture

CISC и RISC

CISC
Complex Instruction Set Computer

богатые инструкции
пример: x86

RISC
Reduced Instruction Set Computer

load / compute / store
ARM, MIPS, PowerPC, RISC-V

Современный RISC CPU не обязан быть простым.

Современный x86 часто декодирует инструкции во внутренние micro-ops.

x86-64 и ARM64

x86 / x86-64

8086 → ... → x86-64

CISC ISA

AMD64 = x86-64

Go: GOARCH=amd64

ARM / AArch64

RISC ISA

load/store model

Apple Silicon, Graviton, Ampere

Go: GOARCH=arm64

Разные ISA → разные machine instructions → одна семантика Go

Другие архитектурные линии

MIPS — классическая RISC-архитектура; удобна как учебная load/store модель.

PowerPC / POWER — RISC-линия IBM; Apple, GameCube/Wii, Xbox 360, PS3, серверы IBM.

SPARC — RISC Sun Microsystems; известен register windows.

RISC-V — открытая модульная ISA; расширение A добавляет atomics.

DEC Alpha — 64-bit RISC, интересна слабой memory model.

IBM z/Architecture — mainframe-линия, Go поддерживает GOARCH=s390x.

Itanium / IA-64: другой путь

IA-64 ≠ x86-64

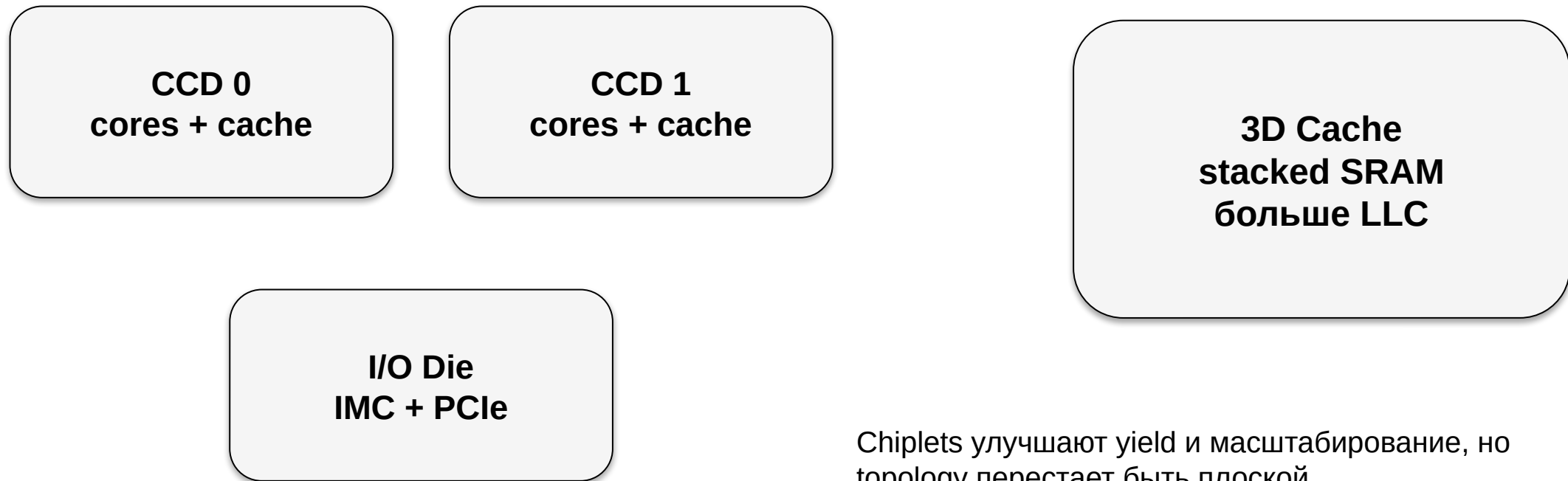
Intel + HP, архитектура EPIC — Explicitly Parallel Instruction Computing.

Идея: больше поиска instruction-level parallelism сделать заранее в compiler.

Проблема: compiler не знает будущие cache miss, branch behavior и runtime data.

Itanium не вытеснил x86-64.

Chiplets и 3D Cache



Chiplets улучшают yield и масштабирование, но topology перестает быть плоской.

3D Cache в основном увеличивает емкость LLC; базовая модель L1/L2/L3 не меняется.

UMA и NUMA

UMA

cores → общий memory controller → RAM
примерно одинаковая стоимость
доступа

NUMA

node 0 ↔ interconnect ↔ node 1
local memory быстрее remote memory

NUMA = Non-Uniform Memory Access

NUMA: local и remote memory



Local memory: доступ через свой memory controller.

Remote memory: запрос проходит через interconnect и другой memory controller.

Remote latency выше и дополнительно расходует bandwidth interconnect.

AMD Infinity Fabric и Intel UPI

AMD Infinity Fabric

**chiplets / memory / coherence / I/O
traffic**

Intel

**UPI между socket
+ internal interconnect / mesh**

Interconnect — транспорт.

NUMA — свойство memory topology.

Chiplet != NUMA node: физическая topology и topology, видимая ОС, не обязаны совпадать.

Возвращаемся к Go

Go code
↓
compiler
↓
ISA
↓
CPU microarchitecture
↓
registers / L1 / L2 / L3
↓
DRAM / NUMA

несколько cores
↓
копии одной cache line
↓
cache coherence
↓
MESI
↓
CAS / atomic contention
↓
sync.Mutex

Следующий блок: cache coherence → MESI → CAS → false sharing → sync.Mutex.